



Análise profunda dos 25 jogos mais promissores. Plano concreto de modificação, mapeamento de dilemas, trade-offs e próximas ações.

1202 JOGOS ANALISADOS • MODS •
DILEMAS SOCIAIS • PSYFUN • UFPE /
UNIVASF

catálogo multi-agente • dashboard em
[psyfun-jogos-research/exports/
dashboard.html](https://psyfun-jogos-research/exports/dashboard.html)

FINALISTAS — 25

jogos candidatos para modificação em pesquisa

PsyFun

Este documento é o catálogo curado dos 25 jogos mais bem posicionados entre os candidatos avaliados para serem **modificados** e operados como instrumentos experimentais pelo grupo PsyFun (UFPE/UNIVASF) em pesquisa sobre dilemas sociais (Prisoner's Dilemma, Public Goods, Stag Hunt, Ultimatum, Dictator, Snowdrift, Trust, Commons, Bargaining, Coordination Public Goods). A fonte primária é **[exports/CONSOLIDADO.jsonl](#)**, gerado pela fase

anterior de pesquisa multi-agente, e as entradas aqui seguem a ordem decrescente de score.

A fórmula de score aplicada foi:

```
score = aderencia_psyfun*2 +  
viabilidade_claude -  
dificuldade*0.5  
      + bonus_popularidade +  
bonus_esforço
```

onde **aderencia_psyfun** e **viabilidade_claude** são notas de 1 a 5 atribuídas pelos agentes especialistas; **dificuldade** é 1-5 (menor é melhor); o bônus de popularidade favorece jogos com comunidades grandes e jam-friendly; o bônus de esforço favorece itens de tamanho XS/S (semanas ao invés de meses). Todos os 25 finalistas têm **aderencia_psyfun=5** e **viabilidade_claude=5**, com **dificuldade=1** ou **2** — ou seja, são os casos onde o esforço-para-primeiro-piloto é mais baixo sem sacrificar validade metodológica.

Para cada jogo abaixo há um perfil técnico, um raciocínio de por que entrou no topo, um plano concreto de modificação em 5 passos que o Claude Code pode executar, o mapeamento de dilemas, os principais trade-offs e a próxima ação recomendada. Quando o JSONL deixou um campo vazio, está marcado como "não informado" — nada aqui foi inventado.

25 FINALISTAS

01 PICO-8 14.50

02 TIC-80 14.50

03 Pyxel 14.50

04 Minetest 14.50

05 Roblox Studio 14.50

06 Prisoner's Dilemma
simulator (direct itch) 14.50

07 Print-and-Play PG games
(itch) 14.50

08 PD visualizations (itch
multi) 14.50

09 Stag Hunt direct FOSS 14.50

10

Ultimatum Game sim (FOSS)**14.50****11 Dictator Game direct
FOSS****14.50****12 Public Goods Game direct
FOSS****14.50****13 Ren'Py Android template
(VN engine)****14.50****14 oTree Prisoner's Dilemma
(sample)****14.50****15 oTree Public Goods Game****14.50****16 oTree Ultimatum Game****14.50****17 oTree Dictator Game****14.50****18 oTree Trust Game (Berg-
Dickhaut-McCabe)****14.50****19 oTree Stag Hunt****14.50****20 Freeciv****14.00****21 Zandronum****14.00**

22 **ioquake3**

14.00

23 **OpenArena**

14.00

24 **LÖVE**

14.00

25 **Widelands**

14.00



PICO-8

Cartridge 2-jogadores local: cada um em split-screen pressiona botão A (cooperar) ou B (trair). Payoff visual em pixels. Template Slingshot-

TL;DR. Fantasy-console minimalista em Lua cuja limitação brutal (cart de 32k) força justamente o tipo de UI binária coop/trair que um experimento clean de dilemas sociais precisa.

Perfil técnico:

- Método de modificação: **creative-mode-in-game** (editor all-in-one embutido).
- Dificuldade: 1 (a mais baixa da escala).

- Esforço estimado: XS (dias, não semanas).
- Viabilidade Claude: 5/5.
- Stack/engine: Lua rodando em fantasy console PICO-8, sprites do editor embutido.
- Plataforma: Windows, Mac, Linux, export Web (HTML5).
- Custo/licença: PICO-8 custa US\$ 15, closed-source mas carts são livres.

Por que entra no topo. O `raciocinio_dificuldade` marca que o editor all-in-one e o Lua simples deixam 99% do workflow amigável para Claude — e a limitação de 32k por cart não é defeito, é feature metodológica. Ela obriga a equipe a remover gambiarra visual e deixar só a decisão binária que o experimento quer medir. O `exemplo_concreto` descreve um cart 2-jogadores local em split-screen onde cada um aperta A (co-

operar) ou B (trair), com payoff visual em pixels, caberia em ~300 linhas de Lua. É um alvo que dá para prototipar numa tarde, rodar no laboratório no dia seguinte e iterar sem atrito.

Como modificar — plano concreto em 5 passos:

1. Baixar PICO-8, estudar o sample "Slingshot" do manual para entender input 2-jogadores local e estrutura de scene (`_init` , `_update` , `_draw`).
2. Criar cart novo com payoff matrix 2×2 hardcoded em tabela Lua e tela de escolha binária por jogador usando botões `btnp(4)` e `btnp(5)` .
3. Implementar 10-20 rodadas iteradas com histórico visual (pontos acumulados em pixels, nenhum número explícito) e tela de reveal simultâneo.

4. Adicionar logging via `printh` para `stdout.p8l` capturando `(round, p1_choice, p2_choice, p1_payoff, p2_payoff, timestamp_frame)`.
5. Exportar HTML5 via `export game.html` para rodar em qualquer notebook do laboratório sem instalar runtime.

Dilemas que mapeia.

- PD — cada rodada é botão A (ficar em silêncio) ou botão B (delatar).
- PG — converter A/B para contribuir X pixels à reserva pública visível no topo.
- SH — escolha entre ícone-cervo (coop) e ícone-lebre (sozinho seguro).
- UG — P1 propõe divisão em 10 pixels, P2 aceita/rejeita.
- DG — P1 aloca pixels, P2 apenas observa.
- TG — P1 envia pixels que triplicam para P2, P2 devolve o que quiser.

- CG — pool comum que regenera por rodada se consumo for moderado.
- CPG — cervo só se ambos apertarem no mesmo tick (coordenação por timing).
- BG — barganha iterada por fatias de pixels.

Trade-offs.

- Licença US\$15 por máquina; closed-source da runtime (mas cart é portátil).
- Estética 128×128 pode parecer "de brinquedo" demais para comitê de ética menos familiarizado com games; precisa ser framing explícito no TCLE.
- Sem multiplayer nativo em rede — 2p é local ou assíncrono via export de cart.

Próxima ação. Comprar uma licença PICO-8 no site Lexaloffle e fazer o cart

"coop/trair" binário em um afternoon
hack antes de decidir se vale o caminho.

Links.

- <https://www.lexaloffle.com/pico-8.php>
-

box/TIC-80

a fantasy computer for making, playing
ing tiny games.

385 Issues
216 Discussions
6k Stars
598 Forks

Mesmo que PICO-8 mas grátis e open; cart 2p
escolha binária

TL;DR. Clone open-source e gratuito do PICO-8 com suporte a Lua, JS, Python, Fennel e Wren — todas as vantagens pedagógicas sem a barreira da licença paga.

Perfil técnico:

- Método de modificação: `creative-mode-in-game` (editor embutido).
- Dificuldade: 1.
- Esforço estimado: XS.

- Viabilidade Claude: 5/5.
- Stack/engine: fantasy console com VM multi-linguagem (Lua padrão, JS/Python/Fennel/Wren opcionais).
- Plataforma: Windows, Mac, Linux, Web, Android, iOS.
- Custo/licença: MIT, 100% grátis.

Por que entra no topo. O `observacoes` é categórico: "MELHOR que PICO-8 pra pesquisa: open + grátis + multi-lingua-ge". Em contexto universitário brasileiro — onde comprar 20 licenças PICO-8 para um laboratório é fricção burocrática real — o TIC-80 remove esse atrito. Mantém o mesmo paradigma de cart único autossuficiente e a mesma disciplina de escassez, com a vantagem de poder escrever o experimento em Python se o pesquisador preferir Python a Lua.

Como modificar — plano concreto em 5 passos:

1. Baixar o binário ou o build web do TIC-80, rodar o cart demo para entender o fluxo `TIC()` principal e o modelo de cart JSON.
2. Clonar o template de PD do PICO-8 (passo 1), portá-lo para Lua-TIC-80 ajustando chamadas (`btnp`, `cls`, `print`, `spr`).
3. Adicionar o parâmetro de `treatment` via menu-de-carga do cart para randomizar entre condições (ex: número de rodadas, visibilidade do histórico).
4. Implementar logging com `trace()` para arquivo externo via wrapper de build.
5. Exportar como HTML5 (`export html`) para deploy em `file://` ou servidor estático da UFPE.

Dilemas que mapeia.

- PD — escolha binária clássica, igual PICO-8 mas grátis.
- PG — contribuição visual à pool comum.
- SH — coordenação cervo/lebre.
- UG — proposta e resposta em duas telas.
- DG — alocação unilateral.
- TG — envio com multiplicador e devolução parcial.
- CG — pool regenerativa.
- CPG — cervo condicional à coordenação temporal.
- BG — barganha iterada.

Trade-offs.

- Comunidade menor que PICO-8 (embora 4k+ stars no GitHub seja saudável).

- Ecossistema de assets/libs mais esparso — mais coisa precisa ser feita do zero.
- Multi-linguagem vira tentação: misturar Lua e Python no mesmo projeto pode dificultar reprodutibilidade; prenda em uma linguagem só.

Próxima ação. Baixar o binário de tic80.com, fazer fork do cart demo mais simples e reimplementar a matriz 2×2 em 100 linhas de Lua para sanity-check antes de escalar.

Links.

- <https://tic80.com>
 - <https://github.com/nesbox/TIC-80>
-

o/pyxel

#03

ame engine for Python

Script Python ~200 linhas: 2p teclado
compartilhado, cada rodada 2 botões. Logging
via CSV

847	18	17k	925
Used by	Discussions	Stars	Fork

TL;DR. Retro game engine 100% Python que remove toda fricção de aprender Lua ou linguagem embedada — Claude escreve, pesquisador edita, pronto.

Perfil técnico:

- Método de modificação: `template-engine` (framework, Claude escreve do zero).
- Dificuldade: 1.
- Esforço estimado: XS.

- Viabilidade Claude: 5/5.
- Stack/engine: Python 3 puro, 16 cores, resolução 256×256.
- Plataforma: Windows, Mac, Linux, Web (via WASM export).
- Custo/licença: MIT, grátis.

Por que entra no topo. Para um laboratório que já tem Python instalado em todo notebook (pandas, psychopy, jupyter), o Pyxel é o menor pulo de ambiente possível. O `exemplo_concreto` cita "~200 linhas Python, 2p teclado compartilhado, cada rodada 2 botões, logging via CSV" — isto é, o pipeline de análise em `pandas` já é nativo do mesmo runtime, sem precisar exportar e reimportar dados. A aderência PsyFun é 5 justamente porque liga o front-end experimental ao back-end estatístico sem serialização intermediária.

Como modificar — plano concreto em 5 passos:

1. `pip install pyxel` em venv dedicado do projeto; rodar `pyxel run pyxel_examples/01_hello_pyxel.py` para validar.
2. Criar `pd_binary.py` com classe `PDGame(pyxel.App)` herdando do pattern `update/draw`; estado `round`, `choices`, `payoffs` como atributos.
3. Implementar input 2p com teclas `A/B` para P1 e `L/K` para P2, lock de decisão ao apertar, reveal simultâneo após ambos.
4. Adicionar `csv.writer` gravando cada rodada em `logs/session_<timestamp>.csv` no diretório do cart; campos conforme schema `docs/03_SCHEMA.md` do projeto PsyFun.
5. Export web com `pyxel package` para rodar no navegador de tablets de

laboratório sem precisar Python instalado no equipamento-sujeito.

Dilemas que mapeia.

- PD — escolha binária A/B, 10 rodadas iteradas.
- PG — slider de contribuição em Python para pool visualizada como barra.
- SH — ícones animais, escolha por click de teclado.
- UG — proposer digita oferta em porcentagem, responder aceita/rejeita.
- DG — P1 aloca, P2 observa.
- TG — envio, multiplicador visual, devolução.
- CG — pool regenerativa por tick.
- CPG — exige decisão sincronizada.
- BG — múltiplas rodadas de oferta-contra-oferta.

Trade-offs.

- Python é lento para 60fps em gráficos complexos — irrelevante para UI experimental simples, mas evite animações pesadas.
- Comunidade menor que Pygame; menos tutoriais para iniciantes absolutos.
- Paleta de 16 cores pode limitar design visual sofisticado caso o estudo precise de estímulos pictóricos complexos.

Próxima ação. Criar um script standalone de 50 linhas implementando a matriz PD canônica ($T=5$, $R=3$, $P=1$, $S=0$) e rodar um teste piloto consigo mesmo e um segundo sujeito antes de qualquer parametrização.

Links.

- <https://github.com/kitao/pyxel>

anti-org/luanti

#04



formerly Minetest) is an open source voxel
creation platform with easy modding and
creation

Mod Lua: chest compartilhado no spawn.
Depositar recurso (PG) vs pegar (trair). Logs via
Lua file IO

utors 1k Issues 13k Stars 2k Forks

TL;DR. Alternativa Minecraft-like open-source com API Lua enorme, multiplayer nativo testado com centenas de jogadores, ideal para experimentos de dilema social embebidos em ambiente 3D imersivo.

Perfil técnico:

- Método de modificação: **mod-loader-oficial** (Lua API estável).
- Dificuldade: 1.

- Esforço estimado: S (uma a duas semanas).
- Viabilidade Claude: 5/5.
- Stack/engine: C++ core, Lua API para mods.
- Plataforma: Windows, Mac, Linux, Android.
- Custo/licença: LGPLv2.1, grátis.

Por que entra no topo. O `observacoes` chama de "CANDIDATO TOP: Lua fácil + multiplayer + 100% free". Diferente de Minecraft, que é hostil a pesquisa (ToS vagos, mods via Forge/Fabric complicados, sem controle de servidor), Minetest é literalmente desenhado para modders, com API estável e documentada. O `exemplo_concreto` mostra um "chest compartilhado no spawn: depositar recurso (PG) vs pegar (trair), logs via Lua file IO" — desenho canônico de Public Goods Game em ambiente naturalístico onde o

sujeito não sabe que está sendo medido pela ação específica.

Como modificar — plano concreto em 5 passos:

1. Instalar Minetest server + client, estudar mods canônicos como `mobs_redo` e `default` para entender a estrutura `mod.conf`, `init.lua`, `depends.txt`.
2. Criar mod `psyfun_commons` com um nó-baú compartilhado (`minetest.register_node`) que aceita depósitos e retiradas por qualquer jogador com log via `minetest.log`.
3. Implementar regra econômica: multiplier automático sobre o conteúdo do baú a cada tick de 60s (simulando retorno do bem público), redistribuído igualmente entre todos players online.

4. Adicionar sistema de chat-macro (`/coop` , `/deposit N`) e HUD customizado mostrando ganhos acumulados visualmente (ícones) sem expor a matemática.
5. Configurar server dedicado do PsyFun (VPS ou máquina do lab) com whitelist de sujeitos, logging estruturado em JSON para pasta `logs/` e sync para análise em pandas.

Dilemas que mapeia.

- PD — par de jogadores em uma câmara com 2 botões-alavanca.
- PG — o baú compartilhado canônico já descrito.
- CG — floresta comum: madeira respawn se corte for abaixo de threshold.
- SH — caçar monstro grande (exige 2p) vs coletar recurso sozinho.

- BG — trading com multiplier assimétrico entre dois jogadores.
- TG — envio de recursos via signpost, retorno à descrição do receptor.

Trade-offs.

- Ambiente 3D é mais imersivo mas também introduz confounds (exploração, perda de direção, efeitos motores) que precisam ser controlados no design.
- Sujeitos precisam de tempo de tutorial não trivial para operar controles; não é adequado para public geral sem familiaridade com games 3D.
- Servidor público pode ter grief — use whitelist e sessões fechadas.

Próxima ação. Subir um servidor local de teste em máquina do laboratório, rodar um mod "hello world" do tutorial

oficial antes de escrever o `psy-fun_commons`.

Links.

- <https://www.minetest.net>
 - <https://github.com/minetest/minetest>
-

#05



Roblox Studio

Experiência custom em 1 semana; deploy zero.

Publishable com 150M+ DAU

ROBLOX

TL;DR. Plataforma criativa com editor visual, Luau scripting e infraestrutura de publicação hosted, permitindo deploy instantâneo para 70M+ de DAU — a opção de maior alcance populacional entre os 25 finalistas.

Perfil técnico:

- Método de modificação:
`creative-mode-in-game` (editor oficial).
- Dificuldade: 1.

- Esforço estimado: M (algumas semanas para produção polida).
- Viabilidade Claude: 5/5.
- Stack/engine: Roblox Engine com Luau (variante de Lua tipada).
- Plataforma: Windows, Mac, Android, iOS, Switch, Xbox, PlayStation.
- Custo/licença: grátis para criar e publicar; Roblox monetiza por Robux.

Por que entra no topo. O `raciocinio_dificuldade` cita "Luau, editor visual, infraestrutura pub hosted, docs massivas" — o conjunto transforma Roblox em um atalho imbatível para levar o experimento do laboratório ao campo. Se o estudo precisa de amostra ecologicamente válida (adolescentes brasileiros gameando em condições naturais), publicar um mundo pequeno no Roblox com consentimento digital faz mais sentido que importar 500 pessoas ao

laboratório. O `observacoes` alerta que pode haver duplicação com outro agente — o score refletiu isso mas decidiu manter pela relevância pura.

Como modificar — plano concreto em 5 passos:

1. Criar conta de desenvolvedor em `create.roblox.com`, instalar Studio e seguir o tutorial oficial "Your First Experience" para entender o fluxo Place/Game/Experience.
2. Usar template "Obby" como base, substituir por uma sala com dois botões `ClickDetector` representando coop/trair; cada clique dispara evento server-side via `RemoteEvent`.
3. Implementar payoff visual com `BillboardGui` mostrando moedas ganhas por rodada; persistir em `DataStoreService` para retornar dados fora do jogo.

4. Configurar compliance: política de privacidade explícita na página da experiência, coleta apenas de ID anônimo (`Player.UserId` hashado SHA-256), consentimento digital antes da primeira rodada.
5. Publicar como unlisted ou private-link para sujeitos recrutados; eventualmente abrir como public com TCLE adaptado.

Dilemas que mapeia.

- PD — dois botões em sala pequena.
- PG — máquina de moedas no centro de arena, alimentada por todos.
- UG — NPC propõe divisão, jogador responde.
- DG — allocator aloca, recipient observa passivamente.
- TG — investir moedas no aliado que escolhe devolução.
- CG — mina compartilhada que esgota.

- CPG — porta que só abre com 2 plates pressionadas simultaneamente.

Trade-offs.

- ToS do Roblox é complexo: coleta de dados de menores exige compliance COPPA e LGPD de menores (art. 14); mesmo anonimizado pode virar problema.
- Dependência de plataforma fechada: se Roblox banir ou mudar ToS, o experimento morre; não é reproduzível em longo prazo.
- IP da Roblox Corp sobre tudo que roda na plataforma; review de comitê de ética vai precisar pronunciar-se sobre titularidade dos dados.

Próxima ação. Antes de qualquer código, pedir parecer do DPO da UFPE sobre viabilidade de coletar dados experimentais em plataforma externa com menores como população-alvo.

Links.

- <https://create.roblox.com>
 - <https://create.roblox.com/docs>
-

Prisoner's Dilemma simulator (direct itch)

Já é PD literal; só adicionar logging e polish

TL;DR. Dezenas de implementações prontas de PD literal no itch.io com source disponível — o candidato de menor esforço possível porque o jogo já é exatamente o experimento.

Perfil técnico:

- Método de modificação: `code-fork` (baixar fonte, adaptar).
- Dificuldade: 1.
- Esforço estimado: XS.

- Viabilidade Claude: 5/5.
- Stack/engine: varia por entry (majoritariamente JS/HTML5).
- Plataforma: Web.
- Custo/licença: gratuito na maioria dos casos.

Por que entra no topo. O `raciocinio_dificuldade` é direto: "vários PD sims prontos no itch com source". Quando a tag tem ~20 entradas e o jogo já é literalmente o instrumento que o pesquisador precisa, a pergunta deixa de ser "como modificar" e passa a ser "adicionar logging e polish visual" — o que é XS em qualquer definição razoável. O `observacoes` grafa "TOP PICK direto".

Como modificar — plano concreto em 5 passos:

1. Listar os ~20 entries em

`itch.io/games/tag-prisoners-dilemma`,

- filtrar pelos que têm source-code aberto (campo "source available").
2. Clonar 3 candidatos, rodar localmente, escolher o que tem arquitetura mais limpa e licença permissiva (MIT ou GPL compatível).
 3. Fork do projeto escolhido em repositório privado do PsyFun; renomear, rebrandizar visualmente, sanitizar assets de terceiros.
 4. Adicionar camada de logging estruturado (JSON via `fetch` para endpoint do lab, ou localStorage com export CSV) e identificação de sujeito por token de sessão.
 5. Deploy em GitHub Pages ou Netlify como URL privada para sujeitos recrutados, ou rodar via `file://` em tablets do laboratório.

Dilemas que mapeia.

- PD — a própria proposição do jogo, já canônica.
- TG — em alguns entries há variantes com confiança iterada.

Trade-offs.

- Qualidade do código varia radicalmente entre entries; muitos são jam code sem testes — auditoria obrigatória antes de produção.
- Autoria: licenças precisam ser checadas individualmente; nem todo itch entry é realmente FOSS.
- Assets visuais podem ser ripados de outros jogos sem atribuição; sanitização visual é requisito.

Próxima ação. Fazer uma varredura manual dos 20 entries em uma sessão de 2 horas, marcar em planilha `{nome, url, licença, linguagem, qualida-`

`de_código}` e voltar com top-3 candidatos concretos.

Links.

- <https://itch.io/games/tag-prisoners-dilemma>
-

Print-and-Play PG games (itch)

Public goods game P&P exata — digitalizar
regras direto

TL;DR. PDFs de regras de Public Goods games prontos para impressão são, na prática, especificações formais diretas — digitalizar é exercício de transcrição, não de design.

Perfil técnico:

- Método de modificação: **template-engine** (regras viram especificação digital).
- Dificuldade: 1.
- Esforço estimado: XS.

- Viabilidade Claude: 5/5.
- Stack/engine: papel+regras como fonte; implementação digital à escolha.
- Plataforma: cross-platform (depende da implementação).
- Custo/licença: varia por título.

Por que entra no topo. O `raciocinio_dificuldade` é lapidar: "Rules PDFs abertos, digitalização trivial". A tag print-and-play no itch tem ~3k jogos, vários deles com mecânicas de contribuição voluntária, confiança, negociação. Quando a regra escrita já serve como especificação formal, o trabalho do Claude vira pegar o PDF, traduzir para estado+ações+payoffs, implementar em qualquer runtime conveniente (oTree, Pyxel, Love2D) e validar contra o papel. O `observacoes` marca "TOP PICK — P&P regras são especificação direta".

Como modificar — plano concreto em 5 passos:

1. Selecionar 3-5 P&P games com mecânica PG explícita (ex: contribuição comum, dilema do bem público), baixar PDFs.
2. Ler as regras em detalhe, transcrever para pseudocódigo estruturado (`rounds`, `players`, `actions`, `payoff_function`) em markdown.
3. Escolher stack-alvo (oTree para rigor experimental, Pyxel para protótipo visual) e implementar uma versão ao pé da letra da regra.
4. Rodar sessões-piloto em papel (mesa de laboratório, 4-6 sujeitos) e comparar decisões com a versão digital, validando paridade de comportamento.
5. Iterar parâmetros (endowment inicial, multiplier, número de

rodadas) para calibrar ao design do estudo PsyFun.

Dilemas que mapeia.

- PG — contribuição voluntária ao bem comum, variando multiplier e número de jogadores.
- UG — subset de P&Ps com negociação de divisão.
- DG — variantes mais simples de alocação unilateral.

Trade-offs.

- Licenças variam; alguns P&Ps são CC-BY, outros são proprietários gratuitos mas não redistribuíveis — trabalho jurídico ponto-a-ponto.
- Papel vs digital: a versão digital pode perder a dimensão social do face-a-face que o papel dá, o que é variável dependente do seu estudo.

- Volume de digitalização acumula se o pesquisador quiser comparar 10 variantes.

Próxima ação. Bater itch.io/physical-games/tag-print-and-play filtrando por "public goods", "cooperation", "commons"; escolher 5 para leitura profunda das regras antes de comprometer código.

Links.

- <https://itch.io/physical-games/tag-print-and-play>
-

#08

PD visualizations (itch multi)

TL;DR. Conjunto de visualizadores de Prisoner's Dilemma e simuladores de torneio estilo Axelrod, todos FOSS, úteis tanto como instrumento quanto como didática para introduzir sujeitos ao jogo.

Perfil técnico:

- Método de modificação: `code-fork` (JS web-native).
- Dificuldade: 1.
- Esforço estimado: XS.

- Viabilidade Claude: 5/5.
- Stack/engine: JavaScript puro.
- Plataforma: Web.
- Custo/licença: gratuito, FOSS.

Por que entra no topo. O `raciocinio_dificuldade` cita "diversos visualizadores de PD/tournament no itch/web, todos FOSS". Particularmente valioso é o link para `axelrod.readthedocs.io` — a biblioteca Python Axelrod é padrão de facto em research de estratégias iteradas, e modificar sua visualização para coletar dados humanos é caminho curto. O `exemplo_concreto` está vazio no JSONL, então o plano abaixo extrapola a partir da natureza do item.

Como modificar — plano concreto em 5 passos:

1. Estudar a biblioteca Python Axelrod e seus visualizadores de torneio; rodar

torneio canônico com TFT, Grudger, Random.

2. Criar fork de um dos visualizadores itch em JS (ou equivalente Python) que permita substituir uma estratégia bot por input humano.
3. Adicionar UI de decisão para sujeito humano rodada-a-rodada, mantendo o display do histórico e do ranking contra bots.
4. Logar as escolhas do sujeito junto com a estratégia adversária presente naquela rodada, timestamp, payoff, estado da reputação.
5. Oferecer protocolo híbrido: treino com bots conhecidos, teste contra estratégia oculta (entre TFT, Grudger, Random, All-D, All-C) para medir adaptação.

Dilemas que mapeia.

- PD — iterado, principal uso.

- TG — variantes com assimetria podem implementar confiança iterada.

Trade-offs.

- Qualidade de código muito variada nos itch entries; Axelrod-lib é sólido mas é back-end, não tem UI amigável pronta.
- Acoplar humano vs bot introduz variável de framing (sujeito pode comportar-se diferente se souber que é bot).
- Menos indicado para coleta de dados em amostra naïve — o jogo original pressupõe alguma familiaridade com a lógica iterada.

Próxima ação. Instalar `pip install axelrod`, rodar um torneio padrão de 1000 rodadas em 5 minutos, e só então avaliar se cabe construir UI por cima ou pegar um fork JS pronto.

Links.

- <https://itch.io/games/tag-prisoners-dilemma>
 - <https://axelrod.readthedocs.io/>
-

The background image is the cover art for the game 'Stag Hunt Direct FOSS'. It features a dark, atmospheric scene with a large, glowing, textured number '#09' in the upper right. Below it, the words 'THE GOATIM' are visible in a stylized, glowing font. The main title 'Stag Hunt direct FOSS' is prominently displayed in the center-left in a large, white, serif font. On the left side, there is a close-up of a stag's head with large, curved antlers, partially obscured by shadows and some red, glowing patterns.

Stag Hunt direct FOSS

Jogo já é SH literal

TL;DR. Implementações FOSS de Stag Hunt no itch já encapsulam a mecânica canônica; basta adaptar visual e logging para rodar como instrumento pronto.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 1.
- Esforço estimado: XS.
- Viabilidade Claude: 5/5.
- Stack/engine: JavaScript.

- Plataforma: Web.
- Custo/licença: gratuito, FOSS.

Por que entra no topo. O `exemplo_concreto` é seco: "Jogo já é SH literal". Stag Hunt é o paradigma de coordenação pura (equilíbrio payoff-dominant vs risk-dominant), e ter uma implementação que já expressa a matriz canônica elimina a etapa mais frágil — traduzir teoria para jogo. O `raciocinio_dificuldade` está vazio no JSONL; a inferência é que a simplicidade é auto-evidente para quem conhece o jogo.

Como modificar — plano concreto em 5 passos:

1. Varrer `itch.io/games/tag-stag-hunt`, identificar 2-3 entries com source disponível e licença clara.
2. Fork do escolhido; rodar localmente; identificar onde está a matriz de payoff e parametrizá-la.

3. Substituir arte por estímulos calibrados para o estudo (ex: ícones animais brasileiros se for estudo regional).
4. Adicionar block de instruction framing controlado (vignette de contexto competitivo vs cooperativo randomizado entre sujeitos).
5. Logging por rodada com `{round, choice, partner_choice, payoff, RT_ms}`.

Dilemas que mapeia.

- SH — coordenação canônica cervo/lebre.
- CPG — extensão com condição de quorum explícita.

Trade-offs.

- Base de entries é pequena (~poucos jogos); qualidade varia.

- Tema "caça" pode ativar considerações éticas em população vegetariana ou sensível — considerar reframing como "projeto conjunto vs projeto individual".
- Sem multiplayer real na maioria dos entries — emparelhamento contra bot pode ser detectado.

Próxima ação. Listar 3-5 entries, testar em 30 minutos, escolher um e fazer fork privado.

Links.

- <https://itch.io/games/tag-stag-hunt>
-

#10

Ultimatum Game sim (FOSS)

TL;DR. Simuladores educacionais de Ultimatum Game em JS ampliam a base FOSS de instrumentos prontos; precisam só de polish visual e logging decente.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 1.
- Esforço estimado: XS.
- Viabilidade Claude: 5/5.

- Stack/engine: JavaScript.
- Plataforma: Web.
- Custo/licença: gratuito, FOSS.

Por que entra no topo. O `exemplo_concreto` e o `raciocinio_dificuldade` estão vazios no JSONL para este entry, mas o pattern é idêntico aos anteriores da série itch-tag: jogo já é o experimento, custo-esforço de modificação é cosmético. Popularidade é descrita como "nicho"; combinado com `observacoes` vazio, é um candidato que funciona como reforço da lista mais que como primeira escolha isolada.

Como modificar — plano concreto em 5 passos:

1. Enumerar entries em `itch.io/games/tag-ultimatum`, identificar FOSS.
2. Fork do mais limpo; testar localmente fluxo proposer → responder → aceita/rejeita.

3. Parametrizar endowment inicial, número de rodadas, visibilidade do histórico (one-shot vs iterado, strangers vs partners).
4. Implementar strategy method opcional (responder pré-especifica aceitação mínima antes de ver oferta), útil para coletar função de aceitação completa em um ensaio.
5. Deploy privado e schema de log alinhado ao `docs/03_SCHEMA.md` do projeto.

Dilemas que mapeia.

- UG — proposta e resposta, canônico.

Trade-offs.

- Base de entries "nicho" = menos opções para cherry-pick.
- Muitos entries educacionais misturam payoff numérico com emoji pedagógico que pode enviesar —

sanitize antes de usar
experimentalmente.

- UG one-shot precisa de amostra grande para estatística decente.

Próxima ação. Testar 2-3 entries e comparar com o sample oficial do oTree (entry 16 desta lista) — se oTree for limpo o suficiente, preferir oTree.

Links.

- <https://itch.io/games/tag-ultimatum>
-



Dictator Game direct FOSS

TL;DR. Implementações diretas de Dictator Game no itch servem como baseline de altruísmo puro — o candidato mais simples de todos, por design.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 1.
- Esforço estimado: XS.
- Viabilidade Claude: 5/5.
- Stack/engine: JavaScript.

- Plataforma: Web.
- Custo/licença: gratuito, FOSS.

Por que entra no topo. Dictator Game é, em termos experimentais, o exercício mais simples possível: uma única decisão unilateral de alocação. `raciocinio_dificuldade` e `exemplo_concreto` estão vazios no JSONL, mas o nível de esforço XS se justifica pela natureza da tarefa — qualquer entry decente já contém 80% do trabalho. `observacoes` também vazio: nada a destacar negativamente.

Como modificar — plano concreto em 5 passos:

1. Ver `itch.io/games/tag-dictator`, selecionar um entry FOSS com UI limpa.
2. Fork; garantir que o fluxo é one-shot (sem trickle de rodadas) se for Dictator puro.

3. Substituir o slider numérico por drag-and-drop de sementes/moedas entre dois sacos para evitar numeracia explícita.
4. Adicionar identificação de recipient: anônimo vs rosto vs nome — variável crítica para manipulação de ingroup/outgroup.
5. Logging binário simples: `{sujeito_id, alocacao_self, alocacao_other, recipient_identity_condition, RT_ms}`.

Dilemas que mapeia.

- DG — alocação unilateral, baseline.

Trade-offs.

- Jogo muito simples; útil só como controle, não como manipulação rica.
- Literatura DG é gigantesca — comparar com meta-análises (ex: Engel 2011) para posicionar achados.

- Sem iteração, sem informação sobre motivações do jogador — só output de alocação.

Próxima ação. Tratar como segunda-escolha atrás do oTree Dictator (entry 17): oTree tem mais rigor metodológico por default.

Links.

- <https://itch.io/games/tag-dictator>
-



TL;DR. Implementações diretas de Public Goods Game no itch fecham a trilogia FOSS-direct com PG canônico pronto para adaptar.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 1.
- Esforço estimado: XS.
- Viabilidade Claude: 5/5.
- Stack/engine: JavaScript.

- Plataforma: Web.
- Custo/licença: gratuito, FOSS.

Por que entra no topo. Mesmo padrão dos três entries anteriores: `raciocinio_dificuldade` e `exemplo_concreto` vazios, mas a proposição é a mesma — jogo-já-é-experimento. Para um pesquisador que quer variar multiplier, número de jogadores, rodadas, punição, ter uma base FOSS pronta poupa o trabalho de escrever o loop principal. `observacoes` vazio.

Como modificar — plano concreto em 5 passos:

1. Varrer `itch.io/games/tag-public-goods`, escolher entry com arquitetura modular.
2. Fork; abstrair parâmetros (`endowment`, `multiplier`, `n_players`, `n_rounds`, `punishment_enabled`) em arquivo de config JSON.

3. Implementar variante com punição (Fehr & Gächter 2000): segundo estágio onde sujeitos pagam para reduzir payoff de free-riders.
4. Adicionar condições experimentais strangers (grupo remixado cada rodada) vs partners (grupo fixo).
5. Exportar dados em formato long compatível com `lmer` em R para análise multinível sujeitos aninhados em grupos.

Dilemas que mapeia.

- PG — contribuição voluntária linear (VCM), canônico.

Trade-offs.

- Nicho edu = base pequena de entries, nem sempre manutenção contínua.
- PG multi-jogador exige coordenação de início de sessão; mais complexo que PD/DG/UG two-person.

- Variantes com punição dobram complexidade de UI e de análise.

Próxima ação. Preferir o sample oficial do oTree (entry 15 desta lista), que já tem literatura publicada apoiando o mesmo código; usar itch como referência visual.

Links.

- <https://itch.io/games/tag-public-goods>
-

py/renpy

Py Visual Novel Engine

#13



VN engine permite criar Public Goods narrativo
do zero — escolhas claras coop/trair com
recompensas visuais

utors 7 Used by 6k Stars 875 Forks

TL;DR. Engine de visual novel MIT-li-
cenciada com scripting simples e export
multi-plataforma, incluindo Android e
iOS — caminho elegante para transfor-
mar um dilema em narrativa experi-
mental com estímulos ricos.

Perfil técnico:

- Método de modificação: `template-
engine` (framework, Claude escreve do
zero).
- Dificuldade: 1.

- Esforço estimado: XS.
- Viabilidade Claude: 5/5.
- Stack/engine: Ren'Py DSL sobre Python.
- Plataforma: Android, iOS, Windows, Linux, Mac.
- Custo/licença: MIT, grátis.

Por que entra no topo. O `exemplo_concreto` aponta o uso ideal: "VN engine permite criar Public Goods narrativo do zero — escolhas claras coop/trair com recompensas visuais". VN é o formato onde o framing contextual da decisão é explícito e controlado (texto, arte, música), útil quando o estudo precisa variar vinheta narrativa como manipulação. O `observacoes` nota honestamente que Ren'Py é engine, não jogo — mas a maturidade (usada em milhares de VNs comerciais) dispensa esforço de estabilização.

Como modificar — plano concreto em 5 passos:

1. Instalar Ren'Py SDK; rodar o tutorial "The Question" para aprender o loop `menu:` com labels e escolhas ramificadas.
2. Criar projeto novo `psyfun_coop_vn`; escrever roteiro de 3-5 cenas que convergem para o dilema, com arte placeholder.
3. Implementar variáveis Python persistentes para track de payoffs acumulados e histórico de escolhas; reutilizar em cenas subsequentes para criar consequências narrativas.
4. Substituir arte placeholder por estímulos pictóricos calibrados (ou CC-BY de Pixabay/OpenGameArt); adicionar trilha sonora controlada para evitar confound emocional.

5. Build para Android APK e Web

HTML5; distribuir por link privado a sujeitos recrutados.

Dilemas que mapeia.

- PD — cena com parceiro preso, escolha delatar/calar.
- UG — proposta de divisão de recurso narrativizada.
- DG — cena de alocação unilateral com NPC recipient.
- TG — envio de recurso a aliado com retorno à descrição.
- BG — barganha narrada em múltiplos turnos.

Trade-offs.

- VN enfatiza narrativa sobre interatividade — pode não ser adequado se o estudo precisa de decisão "fria" sem framing.

- Arte e texto custam tempo humano muito mais que código; tentar reutilizar assets CC-BY para manter escopo viável.
- Público acostumado a VNs é específico (anime-literate), pode enviar amostra.

Próxima ação. Escrever um roteiro curto (500 palavras) com o dilema central antes de abrir Ren'Py — se o texto funcionar em papel, funcionará na engine.

Links.

- <https://www.renpy.org>
 - <https://github.com/renpy/renpy>
-

oTree Prisoner's Dilemma (sample)

Substituir botões Cooperate/Defect por pictogramas (caixa comum vs caixa própria) e mostrar payoffs visuais (frutas, moedas) sem números.

TL;DR. Sample oficial do oTree para PD, ~100 linhas Python/Django, com matrix payoff editável em `Constants` — o candidato de maior rigor metodológico entre os XS de esforço.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 1.
- Esforço estimado: XS.
- Viabilidade Claude: 5/5.

- Stack/engine: Python com Django (oTree framework).
- Plataforma: Web.
- Custo/licença: grátis, open-source.

Por que entra no topo. O `raciocinio_dificuldade` destaca que "Sample app já implementado no otree_library, ~100 linhas. Claude adapta UI trivialmente". oTree é o framework padrão-ouro em experimental economics — usar o sample significa ganhar de graça compliance com práticas da literatura, browser-based deploy, session management, MTurk integration pronta. O `exemplo_concreto` sugere substituição de botões "Cooperate/Defect" por pictogramas (caixa comum vs caixa própria) e payoffs visuais (frutas, moedas) sem números — edição cosmética sobre código metodologicamente sólido.

Como modificar — plano concreto em 5 passos:

1. `pip install otree`, clonar `otree_library`, rodar o sample `prisoner` localmente com `otree devserver`.
2. Editar `Constants` em `models.py` para a matrix canônica do estudo PsyFun (T, R, P, S definidos em `docs/02_REGRAS_DE_NEGOCIO.md`).
3. Substituir templates HTML por versão visual com pictogramas CSS/SVG; remover exposição direta dos números de payoff em favor de ícones acumuláveis.
4. Adicionar page de treino (comprehension check) e page de debrief pós-task com questões de manipulação conferida.
5. Deploy em servidor oTree Hub ou Heroku/Render; coleta de dados

exportável em CSV/Excel nativo do framework.

Dilemas que mapeia.

- PD — canônico, one-shot e iterado ambos triviais.

Trade-offs.

- oTree exige Python + Django + saber deploy web — curva de aprendizado maior que PICO-8/Pyxel no primeiro projeto.
- Visual default é minimalista demais para estudos que precisam de estímulos pictóricos ricos — edição de templates HTML é obrigatória.
- Rodar em lab com rede não internet exige Docker local ou LAN setup.

Próxima ação. Rodar o sample sem modificações em sessão de 2 sujeitos do laboratório para validar pipeline antes de começar a customizar.

Links.

- https://github.com/oTree-org/otree_library
 - <https://otree.readthedocs.io/en/latest/tutorial/prisoner.html>
-



oTree Public Goods Game

Input slider de endowment para pool comum: visualizar como 'árvore compartilhada' que cresce com contribuição coletiva e frutifica para todo

TL;DR. Sample oficial oTree de Public Goods linear (VCM), referência em dezenas de papers, fácil de estender para n rounds, punição, strangers vs partners.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 1.
- Esforço estimado: XS.
- Viabilidade Claude: 5/5.
- Stack/engine: Python/Django (oTree).

- Plataforma: Web.
- Custo/licença: grátis, open-source.

Por que entra no topo. O `raciocinio_dificuldade` destaca parametrização completa de contribuição, multiplicador e redistribuição. O `exemplo_concreto` é particularmente inspirado: "árvore compartilhada que cresce com contribuição coletiva e frutifica para todos; substitui número por animação". Esse tipo de estímulo pictográfico é chave para estudos com população de baixa numeracia, crianças ou contextos cross-cultural onde números abstratos produzem confound.

Como modificar — plano concreto em 5 passos:

1. Clonar oTree library, rodar sample `public_goods` com `otree devserver`.

2. Ajustar `Constants`: `endowment=10`,
`multiplier=1.6`, `n_players=4`,
`n_rounds=10`.
3. Substituir template com animação CSS/SVG da árvore crescente; cada contribuição adiciona folhas à árvore compartilhada no topo da tela.
4. Implementar tratamentos: (a) strangers (re-embaralhar grupos a cada rodada) vs partners (grupo fixo); (b) com/sem punição Fehr-Gächter.
5. Deploy com recrutamento via cartaz + token de sessão; coleta dados em CSV nativo para análise multinível em R.

Dilemas que mapeia.

- PG — VCM clássico, extensões múltiplas.

Trade-offs.

- Grupo de 4 exige coordenação de início de sessão; se algum sujeito cair,

toda a sessão quebra ou vira assíncrona com bot.

- Arte customizada custa horas humanas; ter designer ou usar assets CC-BY.
- Se tratamentos forem muitos, sample size infla rapidamente.

Próxima ação. Decidir cedo se o estudo é within-subjects (todos os tratamentos em cada sujeito) ou between; a arquitetura oTree muda.

Links.

- https://github.com/oTree-org/otree_library/tree/master/public_goods
-

oTree #16 Ultimatum Game

Pizza-slice divisor: proposer divide círculo em duas fatias; responder aceita/rejeita. Visual sem numeracia.

TL;DR. Sample oTree oficial de Ultimatum com proposer/responder em duas páginas, strategy method disponível, pronto para uso metodológico sério.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 1.
- Esforço estimado: XS.
- Viabilidade Claude: 5/5.
- Stack/engine: Python/Django (oTree).

- Plataforma: Web.
- Custo/licença: grátis, open-source.

Por que entra no topo. O `raciocinio_dificuldade` marca a simplicidade de um flow de duas páginas. O `exemplo_concreto` propõe "pizza-slice divisor: proposer divide círculo em duas fatias; responder aceita/rejeita. Visual sem numeracia" — UI perfeita para estudos com crianças ou com sujeitos de baixa escolaridade. `observacoes` lembra que strategy method variant (responder pré-compromete thresholds) já está incluído, o que é padrão-ouro para coletar função de aceitação completa.

Como modificar — plano concreto em 5 passos:

1. Rodar sample `ultimatum` com oTree devserver, familiarizar-se com fluxo proposer → responder → resultado.

2. Parametrizar `endowment` e decidir `direct-response` vs `strategy-method` em `Constants`.
3. Substituir UI numérica por pizza-slice interativo (slider que corta um círculo SVG em duas fatias); responder vê as duas fatias e escolhe aceitar/rejeitar.
4. Implementar block de treino com feedback ("se você rejeitar, ambos ficam com zero") antes da sessão real.
5. Deploy com multi-condição: entre sujeitos, alguns recebem framing "dividindo ganhos" vs "dividindo custos" para manipular loss-aversion.

Dilemas que mapeia.

- UG — canônico, inclui variante strategy method.

Trade-offs.

- Strategy method é mais eficiente estatisticamente mas reduz

engajamento emocional; escolher conforme hipótese.

- UG one-shot precisa de N grande; iterar UG tem problemas de contaminação entre rodadas.
- Pizza-slice visual pode ser menos preciso que slider numérico — validar resolução angular em piloto.

Próxima ação. Rodar piloto com 8 sujeitos em dupla para calibrar tempo médio de sessão e qualidade das respostas no pizza-slice antes de recrutar amostra completa.

Links.

- https://github.com/oTree-org/otree_library/tree/master/ultimatum
-

#17

oTree Dictator Game

Distribuir sementes entre o próprio saco e o saco do parceiro. Interface drag-and-drop, sem exibir números.

TL;DR. App mais simples do oTree: duas páginas, uma decisão, baseline de altruísmo puro e controle natural para UG.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 1.
- Esforço estimado: XS.
- Viabilidade Claude: 5/5.
- Stack/engine: Python/Django (oTree).

- Plataforma: Web.
- Custo/licença: grátis, open-source.

Por que entra no topo. O `raciocinio_dificuldade` é direto: "App mais simples do oTree — 2 pages, 1 decisão". Quando o estudo precisa de controle de altruísmo puro (sem ameaça de rejeição como em UG), DG é o instrumento canônico e o sample oficial é quase zero-esforço. O `exemplo_concreto` propõe drag-and-drop de sementes entre dois sacos — UI inclusiva e sem numeracia.

Como modificar — plano concreto em 5 passos:

1. Rodar sample `dictator` com oTree devserver; familiarizar-se com flow em 10 minutos.
2. Editar `Constants`: `endowment=10 sementes`; decidir identidade do recipient (anônimo vs participante copresente vs caridade nomeada).

3. Substituir slider por UI drag-and-drop entre dois sacos visuais usando HTML5 drag API ou biblioteca `interact.js`.
4. Adicionar manipulação: no grupo-controle, recipient é anônimo; no grupo-tratamento, recipient tem nome e rosto (manipulação de social closeness).
5. Deploy e coleta; export CSV com `{sujeito, alocacao_self, alocacao_other, condition, RT_ms}`.

Dilemas que mapeia.

- DG — alocação unilateral canônica.

Trade-offs.

- Simplicidade extrema pode ser monótona se único task da sessão — palear com UG ou DG variante para sessão mais rica.

- DG one-shot tem teto baixo de informação extraível por sujeito; N grande.
- Sem iteração = sem dinâmica, só snapshot de preferência.

Próxima ação. Usar como warm-up de uma sessão que inclua UG (entry 16): DG dá baseline de altruísmo puro; UG introduz a variável de rejeição; a diferença é informativa.

Links.

- https://github.com/oTree-org/otree_library/tree/master/dictator
-

oTree Trust Game (Berg- Dickhaut- McCabe)

Envio de 'energia' a aliado que amplifica 3x;
aliado decide quanto devolver. Visual: feixe de
luz que cresce ao atravessar o trustee.

TL;DR. Sample oTree do Trust Game canônico (investment multiplicado por 3, return à descrição do trustee), base de centenas de papers em psicologia da confiança.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 1.
- Esforço estimado: XS.
- Viabilidade Claude: 5/5.

- Stack/engine: Python/Django (oTree).
- Plataforma: Web.
- Custo/licença: grátis, open-source.

Por que entra no topo. O `raciocinio_dificuldade` marca "App referência em otree_library. Investment multiplicado por 3, return decision pelo trustee". O Trust Game de Berg-Dickhaut-McCabe é, junto com PD e UG, o tripé experimental da cooperação econômica; ter sample oficial elimina risco de reimplantação divergente. O `exemplo_concreto` sugere visual "feixe de luz que cresce ao atravessar o trustee" — estímulo pictográfico elegante que evita numeracia.

Como modificar — plano concreto em 5 passos:

1. Rodar sample `trust` em oTree devserver.

2. Parametrizar `endowment` e `multiplier` em `Constants` (defaults 10 e 3 são canônicos).
3. Substituir UI numérica por slider/drag do feixe de luz que se amplifica visualmente ao passar pelo trustee; trustee recebe feixe ampliado e escolhe porção a devolver.
4. Implementar variante Trust with Punishment (investor pode punir trustee ao final) como condição between-subjects.
5. Versão multi-round disponível no repo — usar para estudar reputação endógena ao longo de 10 rodadas com partners fixos.

Dilemas que mapeia.

- TG — canônico, direct e com variantes.

Trade-offs.

- Assimetria de papéis (investor vs trustee) exige contrabalanceamento se within-subjects; entre sujeitos é mais limpo.
- Trust with Punishment dobra tempo de sessão e complexidade analítica.
- Multi-round induz efeitos de reputação que podem ou não ser o que se quer.

Próxima ação. Se o estudo PsyFun puser "confiança" como variável central, Trust Game é o instrumento primário; rodar sample local antes de qualquer customização.

Links.

- https://github.com/oTree-org/otree_library/tree/master/trust
-

#19

oTree Stag Hunt

Dois caçadores escolhem cervo (coop alto) ou lebre (sozinho seguro). Ícones animais, escolha por clique.

TL;DR. Sample comunitário oTree de Stag Hunt (matrix game com equilíbrio payoff-dominant vs risk-dominant), útil para estudar coordenação versus cooperação pura.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 1.
- Esforço estimado: XS.
- Viabilidade Claude: 5/5.

- Stack/engine: Python/Django (oTree).
- Plataforma: Web.
- Custo/licença: grátis, open-source.

Por que entra no topo. O `raciocínio_dificuldade` marca "Matrix game com equilíbrio payoff-dominant vs risk-dominant. Simples". Stag Hunt é a ferramenta teórica para separar cooperação (resolver PD) de coordenação (resolver SH) — um resultado negativo em PD e positivo em SH, por exemplo, diz coisas muito diferentes sobre o sujeito. O `exemplo_concreto` é simples: "Dois caçadores escolhem cervo (coop alto) ou lebre (sozinho seguro). Ícones animais, escolha por clique".

Como modificar — plano concreto em 5 passos:

1. Localizar sample stag_hunt no otree_library (fonte na observação oficial).

2. Rodar, confirmar matrix canônica; ajustar payoffs para calibrar gap risk-dominant vs payoff-dominant (Harsanyi-Selten).
3. UI com ícones de cervo e lebre, clique para decidir, reveal simultâneo do parceiro.
4. Pareador: cada sujeito joga 10 rodadas com partner fixo (para reputação) ou strangers (grupo remixado); registrar condição.
5. Adicionar within-subjects condição de comunicação: before-round chat livre vs sem chat — manipulação clássica de coordenação.

Dilemas que mapeia.

- SH — coordenação canônica.
- CPG — com ajuste de quorum para n-players.

Trade-offs.

- Stag Hunt é menos popular que PD na literatura; achados precisam ser contextualizados em relação a ambos.
- Tema caça pode ser insensível; reframe possível como "projeto conjunto ambicioso vs projeto individual seguro".
- SH funciona melhor com 2p; n-players fica mais complicado teoricamente.

Próxima ação. Rodar Stag Hunt pareado com PD (entry 14) no mesmo sujeito (ordem contrabalanceada) para separar coordenação vs cooperação como construto.

Links.

- https://github.com/oTree-org/otree_library
-

eciv/freeciv

#20



s a Free and Open Source empire-building
game inspired by the history of human
n. Upstream repository for...

Ruleset custom onde diplomacia entre 2 civs
reduz a 'declarar guerra vs assinar paz' binário
por turno, loga escolhas; literalmente PD itera

utors 0 Issues 2k Stars 270 Forks

TL;DR. 4X strategy game GPLv2 com rulesets editáveis em texto puro, Lua scripting opcional e multiplayer nativo para até 30 jogadores — plataforma naturalística robusta para dilemas sociais de longa duração.

Perfil técnico:

- Método de modificação: `mod-loader-official`.
- Dificuldade: 2.

- Esforço estimado: S (uma a duas semanas).
- Viabilidade Claude: 5/5.
- Stack/engine: C + GTK, scripting em Lua e rulesets em texto.
- Plataforma: Windows, Mac, Linux, Web (via freeciv-web), Android.
- Custo/licença: GPLv2, 100% grátis.

Por que entra no topo. O `raciocinio_dificuldade` resume o caso: "GPLv2, multiplayer nativo robusto (até 30p), rulesets editáveis em txt puro, web client disponível; PD entre civs é literal". A diplomacia do Freeciv — declarar guerra vs assinar paz — é, estruturalmente, PD iterado. O `exemplo_concreto` propõe um ruleset custom onde essa decisão é capturada como log experimental turno-a-turno. `observacoes` marca "CANDIDATO MUITO FORTE: multiplayer real, PD literal na diplomacia".

Como modificar — plano concreto em 5 passos:

1. Compilar Freeciv do fonte ou usar freeciv-web; rodar partida default single-player para entender turnos e diplomacia.
2. Fork do ruleset `classic` em `data/classic/`, renomear para `psyfun`, simplificar escopo para 2 civs obrigatórias e 50-100 turnos.
3. Escrever script Lua em `data/scenarios/` que instrumenta cada decisão diplomática com log estruturado para arquivo.
4. Configurar server dedicado com autenticação (whitelist de sujeitos), matchmaking 2p a partir do lobby da UFPE.
5. Post-session debrief coletando percepção de adversário, estratégia

percebida, variáveis demográficas;
cruzar com logs comportamentais.

Dilemas que mapeia.

- PD — diplomacia guerra/paz iterada.
- SH — coordenação de alianças em guerra 3-way.
- PG — contribuição para pool de pesquisa cooperativo.
- TG — envio de recursos entre civs aliadas.
- BG — negociação de tratados.
- CG — terreno neutro compartilhado entre civs.

Trade-offs.

- Freeciv tem curva de aprendizado acentuada; sujeitos naïve precisam de treino extensivo antes de sessão real — confound de expertise.
- Sessões longas (60-90 min facilmente); dropout experimental alto.

- Logging em C + Lua exige auditoria cuidadosa para não perder eventos.

Próxima ação. Antes de qualquer mod, rodar partida cheia 2p em modo default para calibrar tempo, identificar onde exatamente capturar a decisão diplomática canônica.

Links.

- <https://www.freeciv.org>
 - <https://github.com/freeciv/freeciv>
 - <https://play.freeciv.org>
-

#21

Zandronum

Mod ACS com pool de munição compartilhada por time: cada kill deposita no pool (coop) ou no estoque pessoal (trair). 16 jogadores simultâneo

TL;DR. Fork GPLv3 de GZDoom focado em multiplayer massivo (64 jogadores), com scripting ACS + DECORATE maduro e servidores públicos ativos — ideal para experimentos coop em ambiente FPS com volume alto.

Perfil técnico:

- Método de modificação: `mod-loader-official`.
- Dificuldade: 2.
- Esforço estimado: S.

- Viabilidade Claude: 5/5.
- Stack/engine: C++ com ACS (Action Code Script) + ZScript/DECORATE.
- Plataforma: Windows, Mac, Linux.
- Custo/licença: engine grátis GPLv3; WAD do Doom original ~R\$10 (ou FreeDoom grátis).

Por que entra no topo. O `raciocinio_dificuldade` destaca "GPLv3, fork GZDoom focado em multiplayer massivo (64p), ACS + DECORATE, servidores públicos; IDEAL pra experimentos coop". O `exemplo_concreto` é vivo: "Mod ACS com pool de munição compartilhada por time: cada kill deposita no pool (coop) ou no estoque pessoal (trair). 16 jogadores simultâneos". Este tipo de dilema embebido em gameplay ativo captura comportamento under pressure, modalidade complementar aos games reflexivos.

Como modificar — plano concreto em 5 passos:

1. Instalar Zandronum + FreeDoom (WAD grátis), rodar modo online local para entender pipeline host/client.
2. Criar mod com ACS que instrumenta pickup de munição: script captura evento `on_pickup`, oferece escolha binária (deposit pool vs personal stash) via menu in-game.
3. Logar cada decisão para arquivo server-side com `{player_id, tick, ammo_type, choice, team_pool_state}`.
4. Implementar balanço para que pool compartilhado seja de-facto benéfico coletivamente mas custoso individualmente (calibrar multiplier).
5. Rodar sessões de 20 min com 8-16 sujeitos recrutados, análise longitudinal da contribuição por rodada.

Dilemas que mapeia.

- PD — pool vs stash em pares.
- PG — contribuição voluntária ao pool de munição do time.
- CG — mapa com recursos compartilhados esgotáveis.
- SH — side-quest exige coordenação de 2+ players para completar.

Trade-offs.

- Público FPS é enviesado demograficamente (predominantemente masculino, jovem, gamer); amostra limitada.
- Violência ambiente pode ser confound emocional; considerar mod "peaceful mode" com ameaças abstratas.
- ACS scripting tem curva de aprendizado; documentação dispersa.

Próxima ação. Confirmar com DPO/comitê de ética se conteúdo violento (ainda que estilizado e low-poly) é aceitável para a amostra-alvo antes de comprometer tempo de mod.

Links.

- <https://zandronum.com>
 - <https://github.com/Torr-Samaho/Zandronum>
-

#22

ioquake3

Fork de cgame/game QVM: mod CTF onde flag compartilhada dá pontos pro time ao deixar no meio (PG) vs pegar pra si (trair). Log de decisões p

TL;DR. Engine GPLv2 do Quake 3 com VM bytecode compilado de C, base de dezenas de mods grandes (Urban Terror, CPMA, Tremulous) — plataforma madura para experimentos FPS com modding sério server-side.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 2.
- Esforço estimado: S.

- Viabilidade Claude: 5/5.
- Stack/engine: C + OpenGL, VM com bytecode QVM compilado de C.
- Plataforma: Windows, Mac, Linux.
- Custo/licença: engine grátis; Quake 3 pago ~R\$25 ou OpenArena grátis.

Por que entra no topo. O `raciocinio_dificuldade` é robusto: "GPLv2, Q3 VM engine, QVM bytecode compilado de C (cgame.qvm/game.qvm), multiplayer 16p+, dezenas de mods grandes; EXCELENTE base". O `exemplo_concreto` descreve fork CTF onde a flag compartilhada dá pontos para o time se deixada no meio (PG) vs pega para si (trair), com log de decisão por evento server-side. A vantagem sobre Zandronum é a base de mods maduros como referência arquitetural.

Como modificar — plano concreto em 5 passos:

1. Clonar `github.com/ioquake/ioq3`, buildar engine e confirmar que cgame/game QVMs compilam com MSVC ou gcc.
2. Fork das QVMs, modificar `g_main.c` e `g_cmds.c` para introduzir o novo evento "depositar flag no centro" vs "escorar pra base".
3. Adicionar logging server-side escrevendo em `qagame.log` a cada decisão de jogador com timestamp e contexto de placar.
4. Rebuild QVMs, empacotar como mod distribuível (`.pk3`), rodar servidor de teste em VPS.
5. Recrutar 8-16 sujeitos por sessão, 20 min de jogo, post-survey e debrief.

Dilemas que mapeia.

- PD — dyadic inside team.

- PG — flag compartilhada como bem público.
- SH — objetivos cooperativos com quorum.
- CG — recursos esgotáveis do mapa.

Trade-offs.

- QVM bytecode exige toolchain antigo; build pode ser fricção no setup.
- Se usar Quake 3 original, custo de assets + titularidade da id Software; OpenArena (entry 23) resolve.
- FPS twitchy exige sujeitos com familiaridade; novatos ficam frustrados.

Próxima ação. Decidir entre ioquake3 + Quake 3 (base clássica) vs OpenArena (assets grátis) — na prática, use OpenArena a menos que haja razão pedagógica específica para Quake.

Links.

- <https://ioquake3.org>
 - <https://github.com/ioquake/ioq3>
-

OpenArena/engine

na modifications to the ioquake3 engine

 45  244  58

utors Issues Stars Forks

Mesmo que ioquake3 mas sem custo de assets

TL;DR. Fork zero-cost do ioquake3 com assets próprios GPL — mesma capacidade técnica, sem custo de licença de assets.

Perfil técnico:

- Método de modificação: `code-fork`.
- Dificuldade: 2.
- Esforço estimado: S.
- Viabilidade Claude: 5/5.

- Stack/engine: C + ioquake3 engine, assets OpenArena GPL.
- Plataforma: Windows, Mac, Linux.
- Custo/licença: 100% grátis, GPLv2.

Por que entra no topo. O `raciocinio_dificuldade` é sintético: "GPLv2, Q3 sem assets Q3 (grátis 100%), fork de ioquake3; mesma capability". O `exemplo_concreto` é literalmente "mesmo que ioquake3 mas sem custo de assets". `observacoes` marca "Zero-cost version do ioquake3". Se o único obstáculo a ioquake3 for o custo dos WADs, OpenArena resolve sem mudar nada mais.

Como modificar — plano concreto em 5 passos:

1. Clonar `github.com/OpenArena/engine`, buildar.
2. Replicar plano de modificação do ioquake3 (entry 22) sobre a codebase

OpenArena — APIs são essencialmente as mesmas.

3. Garantir que o mod funciona tanto com assets OpenArena quanto fallback para Quake 3 original (modularidade assets vs lógica).
4. Mesmo pipeline de logging e recrutamento descrito em 22.
5. Publicar mod em repositório GitHub público do PsyFun com docs bilingues pt-en para maximizar replicação internacional.

Dilemas que mapeia.

- PD — pares intra-time.
- PG — recurso compartilhado.
- SH — coordenação de squad.
- CG — recursos de mapa esgotáveis.

Trade-offs.

- Arte OpenArena é menos polida que Quake 3 — pode afetar imersão.

- Comunidade menor que Quake 3; menos veterans-testers.
- Essencialmente idêntico ao ioquake3 em tudo mais — decisão é só entre assets.

Próxima ação. Escolher OpenArena como default zero-cost; escalar para ioquake3 + Quake 3 apenas se review de assets visuais indicar necessidade.

Links.

- <https://openarena.ws>
 - <https://github.com/OpenArena/engine>
-

e2d/love

#24



an awesome 2D game framework for Lua.

utors 100 Issues 8k Stars 582 Forks

**Template 2p local split-screen ~500 linhas Lua:
cooperar/trair por rodada, payoff visual**

TL;DR. Framework 2D minimalista em Lua licenciado ZLIB, comunidade gigante, ideal para Claude escrever um experimento coop/trair do zero em poucas centenas de linhas.

Perfil técnico:

- Método de modificação: `template-engine` (framework, Claude escreve do zero).
- Dificuldade: 2.
- Esforço estimado: S.

- Viabilidade Claude: 5/5.
- Stack/engine: Lua 5.1 sobre C++ (LuaJIT embutido).
- Plataforma: Windows, Mac, Linux, Android, iOS.
- Custo/licença: ZLIB, grátis.

Por que entra no topo. O `raciocinio_dificuldade` sintetiza: "ZLIB, Lua framework minimal, Claude escreve jogo do zero facilmente; many templates existem". O `exemplo_concreto` descreve "template 2p local split-screen ~500 linhas Lua: cooperar/trair por rodada, payoff visual". LÖVE é o meio-termo entre fantasy console (PICO-8, TIC-80) e engine pesada (Unity, Godot) — tem maturidade e comunidade sem a fricção de console limitations. `observacoes` reconhece: "Framework vs reimpl, mas maturidade+Lua torna IDEAL pra piloto".

Como modificar — plano concreto em 5 passos:

1. `apt install love` (Linux) ou download do site; rodar `love samples/` para validar.
2. Criar `main.lua` com callbacks `love.load`, `love.update(dt)`, `love.draw`, `love.keypressed`; estado `{round, p1_choice, p2_choice, payoffs}`.
3. Implementar UI split-screen horizontal: metade esquerda para P1, direita para P2, com 2 botões-visuais cada e lock ao apertar.
4. Adicionar persistência via `love.filesystem.write` em CSV por sessão com schema canônico.
5. Empacotar como `.love` (zip) para distribuição, ou build Android via `love-android`.

Dilemas que mapeia.

- PD — binário coop/trair.
- PG — contribuição à pool visual.
- SH — cervo/lebre.
- UG — proposer/responder com UI de fatia.
- DG — alocação drag-drop.
- TG — envio com multiplier.
- CG — pool regenerativa.
- CPG — coordenação temporal.
- BG — oferta-contrá-oferta iterada.

Trade-offs.

- Escreve-do-zero = mais trabalho inicial que fork (ex: entries 6, 8-12).
- Lua não é linguagem familiar para todo pesquisador; requer DevOps que saiba Lua para manutenção.
- Sem multiplayer em rede out-of-the-box — LÖVE2D tem libs mas é esforço extra.

Próxima ação. Se a decisão já é Lua (influenciada por PICO-8 ou Minetest estarem na shortlist), LÖVE vale o esforço; se pesquisador prefere Python, Pyxel (entry 3) é rota melhor.

Links.

- <https://love2d.org>
 - <https://github.com/love2d/love>
-



Multi-player com pool comercial: cada jogador pode trade pro aliado (coop) ou acumular (trair).
Log Lua

TL;DR. RTS econômico GPLv2 inspirado em Settlers II, com Lua scripting extensivo e multiplayer nativo — plataforma rica para experimentos de economia compartilhada com 20 anos de maturidade.

Perfil técnico:

- Método de modificação: `mod-loader-oficial`.
- Dificuldade: 2.
- Esforço estimado: S.

- Viabilidade Claude: 5/5.
- Stack/engine: C++ core com Lua scripting.
- Plataforma: Windows, Mac, Linux.
- Custo/licença: GPLv2, 100% grátis.

Por que entra no topo. O `raciocinio_dificuldade` marca "GPLv2, C++ + Lua scripting extensivo, multiplayer nativo, economy deep — similar Settlers/Anno". O `exemplo_concreto` propõe "Multi-player com pool comercial: cada jogador pode trade pro aliado (coop) ou acumular (trair). Log Lua". Widelands é diferente dos FPS: o jogo é economic, decisões são deliberadas, horizonte temporal é longo — captura tipo de cooperação mais próximo de gestão comum de recursos naturalísticos. `observacoes` marca "FORTE: Lua scripting + multiplayer + economia".

Como modificar — plano concreto em 5 passos:

1. Baixar binário de widelands.org, rodar cenário single-player para entender loop econômico (produção, transporte, estoque).
2. Estudar Lua API em [data/scripting/](#) e exemplos em [data/campaigns/](#); criar mapa custom 2-player com biomas balanceados.
3. Escrever script Lua que adiciona "warehouse compartilhado" no centro do mapa; jogadores podem depositar recursos que beneficiam ambos com multiplier.
4. Instrumentar cada depósito/retirada com log estruturado; adicionar tela de "relatório" ao final da partida com métricas de cooperação.

5. Rodar sessões de 30-45 min com pares de sujeitos; post-task survey sobre percepção do parceiro.

Dilemas que mapeia.

- PD — depósito vs acumular em pares.
- PG — warehouse compartilhado.
- CG — recursos naturais esgotáveis do mapa.
- BG — barganha de trade entre jogadores.
- SH — cooperação para completar objetivo grande em dupla.

Trade-offs.

- Curva de aprendizado do RTS é alta; sujeitos naïve precisam treino.
- Sessões longas = dropout; fazer 2 sessões curtas de 30 min melhor que 1 de 60.

- Multiplayer Widelands via internet tem issues ocasionais de sync; preferir LAN do laboratório.

Próxima ação. Se o estudo pedir horizonte temporal longo e decisões deliberadas (vs reflexivas dos FPS), Widelands é candidato forte — mas rode piloto consigo mesmo primeiro para calibrar tempo de sessão.

Links.

- <https://www.widelands.org>
 - <https://github.com/widelands/widelands>
-



Os 25 finalistas acima foram empatados ou muito próximos em score (entre 14.00 e 14.50), o que significa que a decisão de por onde começar não é função da classificação isolada mas do objetivo imediato da próxima fase do projeto PsyFun. Algumas heurísticas de priorização:

- **Se o objetivo for prova-de-conceito rápida em uma semana**, comece por **oTree** (entries 14-19) ou **Pyxel** (entry 3). oTree dá rigor metodológico pronto, Pyxel dá iteração visual rápida em Python puro; qualquer um dos dois leva o laboratório do zero a dados reais coletados em dias.

- **Se o objetivo for alcance populacional (amostra grande, ecológica),** comece por **Roblox Studio** (entry 5) — nenhum outro candidato oferece 70M+ DAU como canal de recrutamento. O custo é de compliance e dependência de plataforma; consulte DPO antes.
- **Se o objetivo for controle experimental máximo com estética custom,** considere **PICO-8** (entry 1) ou **TIC-80** (entry 2). Limitação de cart força minimalismo metodológico e barra confounds visuais — a diferença é custo de licença vs custo de popularidade.
- **Se o objetivo for dilema social naturalizado em ambiente 3D imersivo,** priorize **Minetest** (entry 4) ou **Widelands** (entry 25). Minetest é mais flexível (sandbox, players

constroem); Widelands é mais estruturado (economia com objetivos). Evite Roblox neste subset por questões de compliance.

- **Se o estudo precisar especificamente de UG/DG/TG prontos com rigor**, vá direto ao oTree (entries 16, 17, 18); os equivalentes itch (entries 10, 11) são úteis como referência visual ou backup, não como instrumento primário.
- **Para estudos onde framing narrativo é variável independente, Ren'Py** (entry 13) é a única opção do top-25 desenhada para isso. O custo é horas humanas de roteiro e arte, não horas de código.
- **Evite FPS (entries 21-23) a menos que o estudo requeira decisão sob**

pressão cognitiva ou time-pressure; o público e as considerações éticas específicas (violência, familiaridade com gênero) restringem a amostra.

Recomendação final: rodar dois pilotos em paralelo nas próximas 2-3 semanas, um em **oTree PD** (entry 14) e outro em **Pyxel** (entry 3), ambos XS de esforço. O primeiro estabelece baseline metodológico padrão-ouro; o segundo valida se o pipeline visual custom compensa a extra-fricção. Com dados dos dois pilotos em mão, a próxima decisão arquitetural (escalar em oTree, migrar para Pyxel, ou expandir para Roblox/Minetest) vira empírica em vez de especulativa.